IMPLEMENT CLIENT SCRIPTS &UI POLICY (INCIDENT)

BRAINISTORMING & IDEATION DOCUMENT PHASRE:

Phase 1 – Brainstorming & Ideation

1. Executive Summary• The primary goal of this phase is to explore techniques to improve user experience (UX) and data forms.. During the brainstorming session, the team identified key pain points in manual data entry, such as invalid contact numbers, missing critical information for high-priority incidents, and cluttered forms with non-relevant fields.

2. Problem StatementUsers often submit Incident and Service Request forms with incomplete or improperly formatted data.High-priority tickets are created without adequate business justification.Forms contain too many visible fields at once, leading to confusion and unnecessary cognitive load.No real-time client-side validation exists for formatted inputs (e.g., phone numbers).

3. Brainstorming Ideas & SolutionsDuring the ideation session, the team evaluated several client-side capabilities 4. Final Solution SelectionThe team decided to implement a dual approach leveraging both UI Policies and Client Scripts:UI Policies: Used for simple, no-code dynamic form behavior (Show/Hide, Mandatory, Read-Only).Client Scripts: Used for logic requiring JavaScript execution (e.g., regex pattern validation for phone numbers, on Load alerts, on Submit checks).

REQUIREMENT ANALYSIS DOCUMENT PHAS

Functional & Non-Functional Requirements Specification:

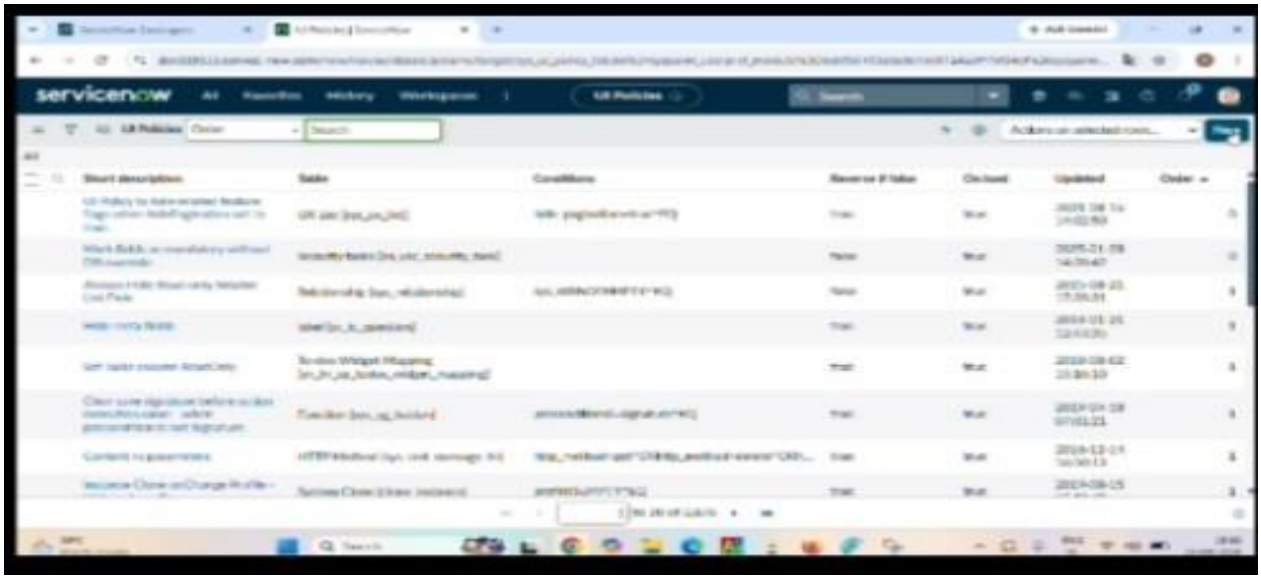
1. IntroductionThis document details the functional and non-functional requirements collected for implementing Client Scripts and UI Policies in Service Now. It serves as a blueprint for configuring client-side rules on the Incident / Service Request forms.

2. Functional Requirements (FR)FR-01: Welcome Banner / NotificationRequirement: Display a welcome message or notification to the user upon opening the form.Mechanism: on Load Client Script.Trigger Event: Form load.FR-02: Conditional Justification for High PriorityRequirement: If Priority is changed to High, display the “Justification for High Priority” field and set it to Mandatory. If Priority is Medium or Low, hide this field and make it Non Mandatory.Mechanism: UI Policy (Condition: Priority == High).FR-03: Category-Based Field VisibilityRequirement: If Category is selected as Hardware, display the “Asset Tag” and “Model Number” fields. Otherwise, keep them hidden.Mechanism: UI Policy FR-04: Phone Number Format ValidationRequirement: Validate that the entered phone number contains exactly 10 numerical digits. If invalid, display an error message and clear the

field.Mechanism: on Change Client Script applied to the Contact Number field.FR-05: Read-Only Form on State ClosureRequirement: When the record state transitions to Closed, set all form fields to Read-Only.Mechanism: UI Policy (Condition: State = Closed).

3. Non-Functional Requirements (NFR)Performance: Client-side scripts must execute efficiently without noticeable UI lag (response time < 200ms).Usability: Form fields must update dynamically without requiring page refreshes.Maintainability: Prefer UI Policies over Client Scripts whenever possible to reduce custom code maintenance.Compatibility: Scripts and UI Policies must work seamlessly across supported web browsers (Chrome, Firefox, Edge).

4. Requirements Traceability Matrix (RTM) ID Description Solution Type Execution PointStatus5. Display Welcome Alert Client Script on Load Approved6. Show/Mandatory Justification Policy UI Policy Condition Match Approved7. FR-03Show Asset Tag on Hardware Category UI Policy Condition Match Approved10-Digit Phone Validation Client scripts Change Approved8. Read-Only on Policy Condition UI Policy Condition Match Approved



PROJECT DESIGN PHASE

1.Introduction:

The Design Phase is a crucial stage of the project where the structure, architecture, and behavior of the Client Scripts and UI Policies in the ServiceNow Incident module are planned and designed before implementation. This phase ensures that the user interface meets all functional requirements efficiently and provides a seamless user experience.

2.Design Objectives:

❖ The main objectives of the design phase are:

- To define the structural logic for Client Scripts (onLoad, onChange, onSubmit) in the Incident form.
- To design UI Policies that dynamically control form fields without writing complex code.
- To determine mandatory, visible, and read-only field rules based on specific form conditions.
- To ensure optimal form performance and fast client-side execution.

To structure clear user interaction flows within the ServiceNow Incident management interface.

3.Design Process:

The Incident form layout and functional requirements were thoroughly analyzed to plan the execution flow. The design outlines how Client Scripts will handle client-side validations and event-driven actions (such as form load or field value changes). Additionally, UI Policy rules and UI Policy Actions were structured to dynamically hide, show, mandate, or lock fields based on defined conditions on the Incident form.

4.Design Outcome:

The architecture and design for the Client Scripts and UI Policies were successfully prepared for implementation in the Incident module. Clear conditions, trigger points, and field behavior rules have been established to guide the development phase effectively.

PROJECT PLANNING PHASE

1. Project Overview:The main objective of this project is to implement Client Scripts and UI Policies in the ServiceNow Incident module. These configurations will help control the behavior of incident forms, improve user interaction, and ensure that required fields and form actions work according to the defined business requirements.

2. Project Objectives:

• To configure Client Scripts for dynamic form behavior.

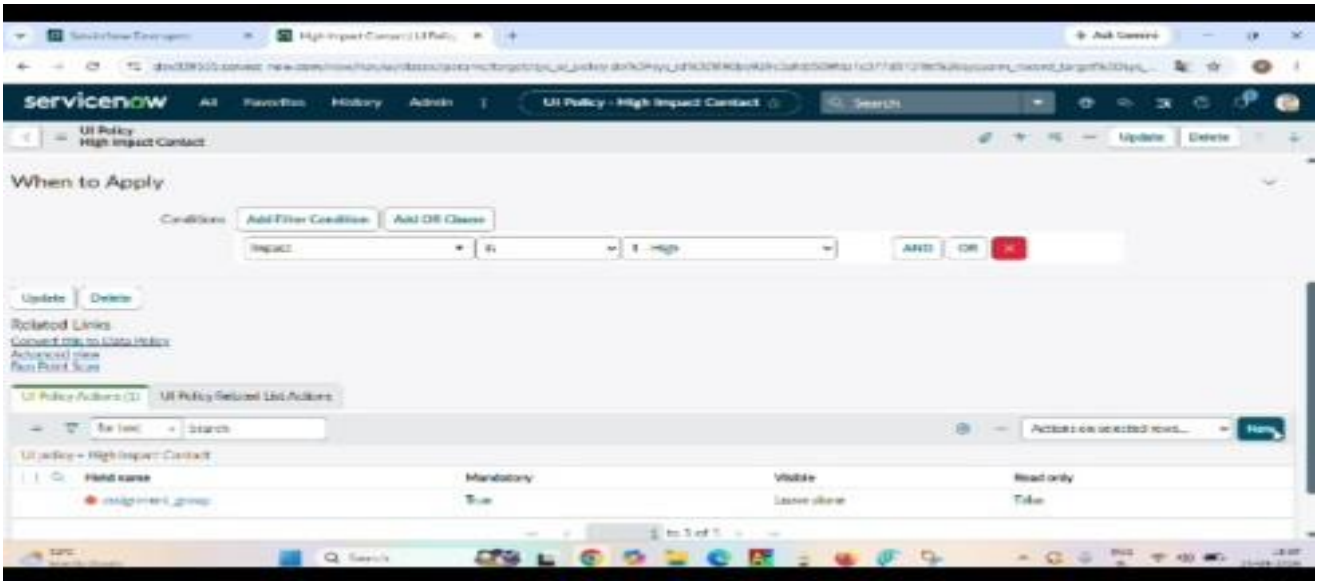
• To implement UI Policies for controlling field visibility, mandatory status, and read-only conditions.

• To improve the usability of the Incident form.

- To reduce incorrect or incomplete data entry.
- To ensure the Incident form follows the required business rules.

3. Planning Activities:

The planning phase includes identifying the Incident form requirements, deciding which fields need Client Script logic, and determining the conditions for UI Policies. The required configurations, testing approach, and expected results will also be planned before implementation.



4.Resources Required:

- ServiceNow development instance
- Incident application/module
- Client Script configuration
- UI Policy configuration
- Test records for validation

5.Expected Outcome:

After successful implementation, the Incident form should respond dynamically based on the defined conditions. Fields should become visible, hidden, mandatory, or read-only as required. This will provide a more consistent and efficient Incident management process.6. Project Timeline:The project will be completed through the

following phases:The planning phase provides the foundation for implementing and testing the Client Scripts and UI Policies successfully.

PROJECT DEVELOPMENT PHAS

1.Development Overview:

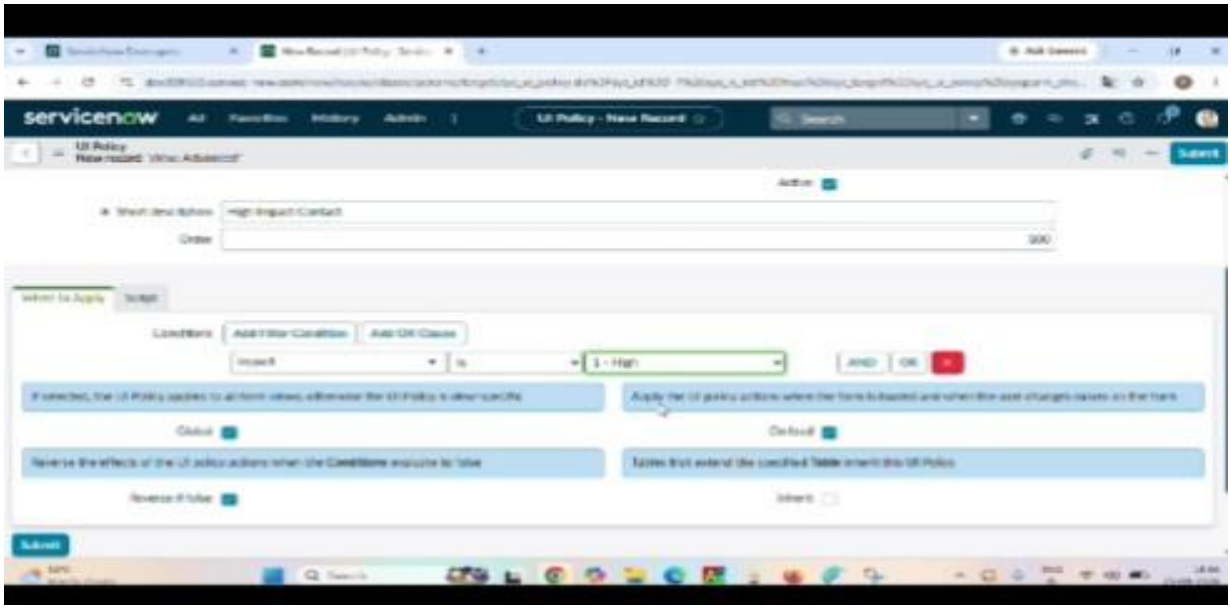
The development phase focuses on implementing Client Scripts and UI Policies in the ServiceNow Incident application. These configurations are used to improve the Incident form by controlling field behavior, validating user inputs, and dynamically changing the form based on user actions.

2. Client Script Development:

Client Scripts are developed to perform client-side actions when users interact with the Incident form. The required scripts are configured based on the project requirements.

❖The Client Script can be used to:

- Validate Incident form inputs.
- Automatically populate or modify field values.
- Display messages when incorrect information is entered.
- Control field behavior based on user selections.
- Execute actions when the form is loaded, changed, or submitted.
- The scripts are tested on the Incident form to ensure that the expected behavior occurs without affecting other functionality.



3. UI Policy Development:

➢ UI Policies are configured to dynamically control the Incident form fields without requiring complex scripting. Based on the specified conditions, fields can be made mandatory, visible, or read-only.

➢ For example, a UI Policy can be created to make a particular Incident field mandatory when a specific condition is satisfied. UI Policy Actions are then configured for the required fields.

PROJECT TESTING PHASES

1.Introduction:

The Testing Phase is an important stage of the project where the implemented Client Scripts and UI Policies in the ServiceNow Incident module are tested to ensure that they work correctly according to the defined requirements.

2.Testing Objectives:

❖The main objectives of testing are:

- To verify that the Client Scripts work as expected.
- To check whether UI Policies correctly control form fields.
- To ensure that mandatory, visible, and read-only field conditions are working properly.

- To identify and fix any errors or unexpected behavior.
- To ensure that the Incident form provides the expected user experience.

3.Testing Process:

The Incident form was tested with different field values and conditions. The Client Script was executed by performing the required actions on the Incident form. The UI Policy was also tested to verify whether the specified fields became visible, hidden, mandatory, or read-only based on the configured conditions.

4.Testing Result:

The implemented Client Script and UI Policy were tested successfully in the Incident module. The configured conditions and field behaviors worked as expected. No major functional issues were identified during testing.